This line ensures that when the script is run, the `calculator()` function is called, starting the main loop.

Step 5: Handling Edge Cases

When working with calculators, it's important to handle potential issues that could arise from user input or the nature of the operations. Here are some edge cases and how we handle them:

- **Division by zero**: As mentioned, the `divide` function checks if the second number is zero and returns an error message if so.
- **Invalid inputs**: If the user enters anything other than a number when prompted for the values, the program catches it with the `ValueError` exception and asks them to enter valid numbers again.
- **Invalid menu choices**: If the user selects an option that doesn't exist (other than 1-5), the program will notify them of the invalid input and prompt them again.

Conclusion

This simple calculator project demonstrates basic Python concepts such as functions, loops, user input, and error handling. It's a great starting point for beginners who want to practice these essential skills. You can extend this project further by:

- Adding more advanced mathematical operations (like exponentiation, square roots, etc.).
- Implementing a graphical user interface (GUI) using Python libraries such as Tkinter.
- Storing previous results and allowing the user to perform operations using those results.